

WorkBuddy

CODESYS 编程能力技能清单与安装指南

本文档详细列出 WorkBuddy 编写 CODESYS PLC
代码所依赖的全部技能（Skill），

包含技能名称、功能描述、知识覆盖范围及多种安装方式，

使新安装 WorkBuddy 的电脑也能快速具备相同的 CODESYS 开发能力。

文档版本	v1.0
生成日期	2026-07-02
适用平台	Windows / macOS / Linux
技能数量	2 个（1 核心 + 1 辅助）

一、概述

WorkBuddy 的 CODESYS PLC

编程能力并非凭空生成，而是由安装在用户级别的技能（Skill）所驱动。技能是 WorkBuddy 的知识扩展包，包含领域知识、代码模板、设计模式和最佳实践。当用户提及"CODESYS"、"PLC"、"ST"、"凸轮表"、"编码器"、"卷绕"、"功能块"等关键词时，WorkBuddy 会自动加载相应技能，从而生成符合 IEC 61131-3 标准的结构化文本（ST）代码。

技能架构总览

类别	技能名称	角色	来源
核心技能	codesys-development	CODESYS ST 编程核心知识：语法规则、FB 设计模式、凸轮表算法、代码模板	AI 创建 (手动安装)
辅助技能	mermaid-diagram	流程图/状态机/架构图生成，用于 PLC 逻辑可视化与文档化	SkillHub 市场 (市场安装)

核心技能 codesys-development 提供了 CODESYS 编程所需的全部领域知识，包括 IEC 61131-3 ST 语法、功能块（FB）设计模式、凸轮表数据采集与曲线拟合算法、编码器/卷绕轴处理、边沿触发控制逻辑、错误处理与状态机等。这是编写 PLC 代码的直接知识来源。

辅助技能 mermaid-diagram 提供了 Mermaid 图表生成能力，可将 PLC 逻辑转化为流程图、状态机图、时序图等可视化形式，便于设计评审和文档化。虽然不直接生成 ST 代码，但在 PLC 程序设计和沟通中起到重要的辅助作用。

二、核心技能：codesys-development

2.1 基本信息

属性	值
技能名称	codesys-development
存储位置	~/.workbuddy/skills/codesys-development/
技能层级	用户级（User-level），所有项目共享
创建方式	AI 创建（通过 skill-creator 或手动编写）
市场来源	无（非市场技能，需手动安装）
触发关键词	CODESYS, PLC, ST, 凸轮表, 编码器, 卷绕, 功能块, FB, IEC 61131-3

2.2 知识覆盖领域

领域	具体内容
IEC 61131-3 ST 语法	变量声明（VAR_INPUT/OUTPUT/IN_OUT/VAR/CONSTANT）、数据类型（BOOL/INT/LREAL/ARRAY）、边沿检测模式、RETURN 语句、注释语法
功能块设计模式	模式A：数据采集 FB（外部触发、数组填充、边沿启停、双向检测）；模式B：数据处理 FB（VAR_IN_OUT 数组传递、bExecute 上升沿触发、错误码）；模式C：状态机 FB（CASE 状态切换、IDLE/RUNNING/DONE/ERROR）
凸轮表专项知识	凸轮表 = (X, Y) 数据对、角度触发采集、平均斜率线性拟合、移动平均平滑、指数平滑、数组设计（编译期最大 10000）、SoftMotion 集成
变量命名规范	b=BOOL, n=INT, r=REAL, arr=ARRAY, Step=步进, Max=上限, Last=上一周期, Next=下一目标, Actual=当前有效, Fit=拟合结果
错误处理模式	ERR_OK/ERR_FULL/ERR_PARAM 常量定义、参数校验提前返回、错误码输出引脚
FB 实例化与调用	声明 FB 实例、VAR_IN_OUT 数组绑定、带参数调用语法

2.3 包含的文件

文件路径	说明
SKILL.md	技能主文件，包含概述、知识领域、使用流程、限制说明
references/fb-templates.md	3 个即用型 FB 代码模板：数据采集 FB、数据处理 FB、测试程序 PRG

文件路径	说明
references/cam-table-patterns.md	凸轮表设计模式与算法：采集模式、双向检测、线性拟合、移动平均、指数平滑、质量评估
references/st-code-snippets.md	10 类常用 ST 代码片段：边沿检测、数组初始化、移动平均、限幅、状态机、定时器、类型转换、循环缓冲区、错误码、VAR_IN_OUT 最佳实践

2.4 安装方式

方法一：手动复制（推荐）

由于 codesys-development 是 AI 创建的技能，不在 WorkBuddy 推荐市场中，最可靠的方式是将技能文件直接复制到目标电脑的技能目录：

```
# Windows
1. ■ codesys-development ■■■■■■■■
   C:\Users\&lt;■■■&gt;\.workbuddy\skills\codesys-development\

# macOS / Linux
2. ■ codesys-development ■■■■■■■■
   ~/.workbuddy/skills/codesys-development/

# ■■■■■■
~/.workbuddy/skills/codesys-development/
  ■■■ SKILL.md
  ■■■ references/
    ■■■ fb-templates.md
    ■■■ cam-table-patterns.md
    ■■■ st-code-snippets.md
```

注意：本文档附带的 skill-bundle 文件夹中已包含完整的技能文件，可直接复制使用。

方法二：通过 skill-creator 重建

在新电脑上安装 WorkBuddy 后，使用内置的 skill-creator 技能重建：

[illegible]

方法三：通过对话搜索市场确认

在 WorkBuddy 对话中输入以下内容，确认市场是否已有同类技能：

```
# ■ WorkBuddy ■■■■■■
■■ codesys ■■
# ■
■■ PLC ■■■■■■ skill
```

若市场已有 codesys 相关技能, 可直接安装; 若无, 则使用方法一或方法二。

三、辅助技能：mermaid-diagram

3.1 基本信息

属性	值
技能名称	mermaid-diagram
存储位置	~/.workbuddy/skills/mermaid-diagram/
技能层级	用户级（User-level），所有项目共享
创建方式	市场安装（来源：SkillHub）
市场 slug	mermaid-diagram
版本	1.0.0
触发关键词	画图, 流程图, 架构图, 时序图, 脑图, mermaid, diagram, flowchart

3.2 在 CODESYS 开发中的作用

mermaid-diagram 技能在 CODESYS 开发流程中承担可视化与文档化角色：

使用场景	Mermaid 图类型	说明
PLC 程序逻辑流程	Flowchart (flowchart TD)	将 FB 内部逻辑转化为流程图，便于评审
状态机设计	State Diagram (stateDiagram-v2)	可视化 IDLE/RUNNING/DONE/ERROR 状态转换
系统架构	Flowchart LR / C4	展示 PLC、编码器、伺服、HMI 之间的连接关系
通信时序	Sequence Diagram	展示主从轴同步、CamIn 启动时序
功能分解	Mindmap	功能块树形结构分解

3.3 安装方式

方法一：通过 WorkBuddy 推荐市场安装（推荐）

mermaid-diagram 来自 SkillHub 市场，可直接在新电脑上搜索安装：

```
# ■ WorkBuddy ■■■■■■■■■■■■
■■ mermaid-diagram ■■
# ■
■■■■ mermaid ■■■■
# ■
find skill mermaid
```

方法二：手动复制

```
# ■ mermaid-diagram ■■■■■■■■■■
~/.workbuddy/skills/mermaid-diagram/
■■■ SKILL.md
■■■ _meta.json
■■■ _skillhub_meta.json
■■■ references/
    ■■■ diagram-patterns.md
```

四、完整安装指南（新电脑操作步骤）

4.1 前置条件

1. 目标电脑已安装 WorkBuddy 桌面版并完成首次启动。
2. 已完成 WorkBuddy 账号登录。
3. 准备好本文档附带的 skill-bundle 文件夹（含两个技能的完整文件）。

4.2 安装步骤

步骤	操作	说明
1	安装 mermaid-diagram（市场安装）	在 WorkBuddy 对话中输入"安装 mermaid-diagram 技能", 从市场直接安装
2	准备 codesys-development 文件	从本文档附带的 skill-bundle 文件夹中取出 codesys-development 目录
3	复制到技能目录	将 codesys-development 文件夹复制到 ~/.workbuddy/skills/ 目录下
4	重启 WorkBuddy	关闭并重新打开 WorkBuddy, 使其加载新技能
5	验证安装	在对话中输入"帮我写一个 CODESYS 功能块", 确认技能已生效

4.3 验证清单

验证项	验证方法	预期结果
codesys-development 已加载	对话输入"写一个 CODESYS 凸轮表数据采集 FB"	生成包含 VAR_IN_OUT、边沿触发、双向检测的完整 FB 代码
FB 模板可用	对话输入"给我一个 CODESYS 数据处理功能块模板"	生成包含 bExecute 上升沿触发、移动平均计算的 FB 代码
凸轮表知识可用	对话输入"CODESYS 凸轮表怎么做线性拟合"	解释平均斜率拟合公式 $FitX[i] = X[0] + AvgSlope * (Y[i] - Y[0])$
mermaid-diagram 已加载	对话输入"画一个 PLC 状态机流程图"	生成 mermaid stateDiagram-v2 代码块

五、技能文件结构与路径速查

5.1 完整目录结构

```
~/workbuddy/skills/
├── codesys-development/          # [CODESYS ST]
│   ├── SKILL.md                 # frontmatter +
│   └── references/
│       ├── fb-templates.md      # 3 FB
│       ├── cam-table-patterns.md #
│       └── st-code-snippets.md  # 10 ST
│
├── mermaid-diagram/            # [Mermaid]
│   ├── SKILL.md                 #
│   ├── _meta.json               #
│   ├── _skillhub_meta.json      #
│   └── references/
│       └── diagram-patterns.md  #
│
# Windows
C:\Users\<>\workbuddy\skills\

# macOS / Linux
/home/<>\workbuddy/skills/  /Users/<>\workbuddy/skills/
```

5.2 SKILL.md frontmatter 格式说明

每个技能的 SKILL.md 文件以 YAML frontmatter 开头，包含 name 和 description 两个字段。WorkBuddy 通过 description 中的关键词来决定何时加载该技能：

```
---
name: codesys-development
description: CODESYS IEC 61131-3 Structured Text (ST) development skill.
  Use when: (1) writing PLC programs in CODESYS, (2) creating function
  blocks (FB) for motion control, cam tables, encoder processing, (3)
  implementing winding/tension control logic, (4) generating cam table
  data acquisition and curve fitting algorithms, (5) any mention of
  "CODESYS", "PLC", "ST", "CODESYS", "CODESYS", "CODESYS", "CODESYS", "FB".
  Covers FB design patterns, VAR_IN_OUT array binding, edge-triggered
  control, bidirectional detection, and error handling.
---
```

5.3 技能能力边界

能力	说明
支持的输出	IEC 61131-3 结构化文本（ST）代码、功能块（FB）、程序（PRG）、代码模板
不支持的可视化	FBD（功能块图）、LD（梯形图）、SFC（顺序功能图）- 仅支持 ST 文本
不能编译/测试	无法直接编译或运行 CODESYS 代码，需手动复制到 CODESYS Development System
数组大小限制	编译期最大 10000 元素，运行时通过 MaxArrayLength 参数配置

附录 A：codesys-development 技能完整内容

以下为 codesys-development 技能的 SKILL.md 完整内容，可用于手动创建或通过 skill-creator 重建该技能。

A.1 SKILL.md

```
---
name: codesys-development
description: CODESYS IEC 61131-3 Structured Text (ST) development skill. Use when: (1) writing PLC programs i
---

# CODESYS Development Skill

## Overview

This skill provides structured knowledge for writing CODESYS IEC 61131-3 ST code, focusing on:
- Function Block (FB) design patterns for industrial motion control
- Cam table data acquisition and curve fitting
- Encoder/winding axis processing
- Edge-triggered control logic
- Error handling and state machines

## Core Knowledge Areas

### 1. IEC 61131-3 ST Syntax Rules

- Variable declarations: `VAR_INPUT`, `VAR_OUTPUT`, `VAR_IN_OUT`, `VAR`, `VAR_CONSTANT`
- Data types: `BOOL`, `INT`, `UINT`, `DINT`, `LREAL`, `STRING`, arrays
- `ARRAY[0..N] OF LREAL` – fixed-size arrays (CODESYS does not support dynamic arrays)
- `VAR_IN_OUT` for passing large arrays by reference (avoid memory copy)
- `FUNCTION_BLOCK ... END_FUNCTION_BLOCK` structure
- `PROGRAM ... END_PROGRAM` structure
- Edge detection pattern: `IF bEnable AND NOT LastEnable THEN ... LastEnable := bEnable;`
- `RETURN` statement for early exit
- Comments: `//` single line, `(* ... *)` block

### 2. Function Block Design Patterns

#### Pattern A: Data Acquisition FB
- Trigger source: external signal (angle, position, or time)
- Array fill pattern: index increment with overflow check
- Edge-triggered start/stop
- Auto-read current value as start point
- Bidirectional movement detection

#### Pattern B: Data Processing FB
- Takes source arrays via `VAR_IN_OUT`
- Outputs result arrays via `VAR_IN_OUT`
- Execute trigger with `bExecute` rising edge
- Single-cycle or multi-cycle calculation
- Error codes for invalid input

#### Pattern C: State Machine FB
- `CASE state OF` pattern
- States: IDLE, RUNNING, DONE, ERROR
- Explicit state transitions
- Reset capability

### 3. Cam Table Specific Knowledge

- Cam table = array of (X, Y) pairs mapping master axis position to slave axis position
- X-axis: typically encoder position or master axis position
```

- Y-axis: typically slave axis position or angle
- Trigger by angle interval: every N degrees of winding axis, capture encoder position
- Average slope = $(X[\text{end}] - X[0]) / (Y[\text{end}] - Y[0])$
- Linear fit: $\text{FitX}[i] = X[0] + \text{AvgSlope} * (Y[i] - Y[0])$
- Moving average smoothing: average of N neighboring points (N must be odd)
- Array max size: 10000 (compiled), runtime configurable via MaxArrayLength

4. Common Variable Naming Conventions

Prefix	Meaning	Example
`b`	BOOL	`bEnable`, `bDone`, `bError`
`n`	INT/UINT	`nState`, `nCount`
`r`	REAL/LREAL	`rPosition`, `rAngle`
`arr`	ARRAY	`arrCamTableX`
`Step`	Configurable step	`StepInterval`
`Max`	Maximum limit	`MaxArrayLength`
`Last`	Previous cycle value	`LastEnable`, `LastAngle`
`Next`	Next target	`NextTargetAngle`
`Actual`	Current valid	`ActualLength`
`Fit`	Fitted result	`FitLineX`, `FitSmoothY`

5. Error Handling Pattern

```

```iecst
VAR CONSTANT
 ERR_OK : UINT := 0;
 ERR_FULL : UINT := 1; // Array full
 ERR_PARAM : UINT := 2; // Invalid parameter
END_VAR

// At start of logic:
IF StepInterval <= 0 THEN
 bError := TRUE;
 ErrorCode := ERR_PARAM;
 RETURN;
END_IF;
```

```

6. FB Instantiation and Calling

```

```iecst
// Declaration
VAR
 MyFB : FB_CamTableGenerator;
 arrX : ARRAY[0..9999] OF LREAL;
 arrY : ARRAY[0..9999] OF LREAL;
END_VAR

// Calling with VAR_IN_OUT arrays
MyFB.bEnable := TRUE;
MyFB.EncoderPosition := rEncPos;
MyFB(CamTableX := arrX, CamTableY := arrY);
```

```

Reference Files

- `references/fb-templates.md` – Ready-to-use FB code templates
- `references/cam-table-patterns.md` – Cam table design patterns and algorithms
- `references/st-code-snippets.md` – Common ST code snippets and utilities

Usage Workflow

1. User describes the PLC function needed
2. Identify which pattern applies (acquisition, processing, state machine)

3. Generate FB code using the appropriate template
4. Generate a test PRG program
5. Provide usage instructions and parameter tuning guidance

Limitations

- Cannot directly compile or test CODESYS code (no CODESYS runtime access)
- Code must be copied manually into CODESYS Development System
- Array sizes are fixed at compile time (max 10000 elements)
- Does not support visualization (FBD/LD) – ST only

A.2 references/fb-templates.md

```
# CODESYS FB Templates

## Template 1: Data Acquisition Function Block

```iecst
FUNCTION_BLOCK FB_DataAcquisition

VAR_INPUT
 bEnable : BOOL; // Start/stop control (edge-triggered)
 bReset : BOOL; // Reset all data
 SensorValue : LREAL; // Input signal to record
 TriggerSource : LREAL; // Signal that triggers capture (e.g., angle)
 MaxArrayLength: UINT := 1000; // Max capture points
 StepInterval : LREAL := 1.0; // Trigger interval
 bUseCurrentStart: BOOL := TRUE;
 ManualStart : LREAL; // Manual start value (if bUseCurrentStart=FALSE)
END_VAR

VAR_OUTPUT
 bRunning : BOOL;
 bDone : BOOL;
 bError : BOOL;
 ErrorCode : UINT;
 ActualLength : UINT;
END_VAR

VAR
 StartValue : LREAL;
 NextTarget : LREAL;
 LastEnable : BOOL;
 IsFirstPoint : BOOL;
 MoveDir : LREAL;
 LastTrigger : LREAL;
END_VAR

VAR CONSTANT
 ERR_OK : UINT := 0;
 ERR_FULL : UINT := 1;
 ERR_PARAM : UINT := 2;
 MAX_SIZE : UINT := 10000;
END_VAR

VAR_IN_OUT
 DataArray : ARRAY[0..9999] OF LREAL;
END_VAR

// Reset
IF bReset THEN
 bRunning := FALSE; bDone := FALSE; bError := FALSE;
 ErrorCode := ERR_OK; ActualLength := 0;
 StartValue := 0; NextTarget := 0;
 LastEnable := FALSE; IsFirstPoint := FALSE;
 MoveDir := 1.0; LastTrigger := 0;
 RETURN;
END_IF

// Param check
IF StepInterval <= 0 THEN
 bError := TRUE; ErrorCode := ERR_PARAM; RETURN;
END_IF

IF MaxArrayLength > MAX_SIZE THEN
 MaxArrayLength := MAX_SIZE;

```

```

END_IF

// Edge detection
IF bEnable AND NOT LastEnable THEN
 bRunning := TRUE; bDone := FALSE; bError := FALSE;
 ErrorCode := ERR_OK; ActualLength := 0;
 IsFirstPoint := TRUE; MoveDir := 1.0;
 LastTrigger := TriggerSource;
 IF bUseCurrentStart THEN
 StartValue := TriggerSource;
 ELSE
 StartValue := ManualStart;
 END_IF
 NextTarget := StartValue;
ELSIF NOT bEnable AND LastEnable THEN
 IF bRunning THEN
 bRunning := FALSE; bDone := TRUE;
 END_IF
END_IF
LastEnable := bEnable;

// Capture
IF bRunning THEN
 // Detect direction
 IF TriggerSource > LastTrigger THEN
 MoveDir := 1.0;
 ELSIF TriggerSource < LastTrigger THEN
 MoveDir := -1.0;
 END_IF
 LastTrigger := TriggerSource;

 // First point
 IF IsFirstPoint AND (ActualLength < MaxArrayLength) THEN
 dataArray[ActualLength] := SensorValue;
 ActualLength := ActualLength + 1;
 NextTarget := StartValue + StepInterval * MoveDir;
 IsFirstPoint := FALSE;
 END_IF

 // Subsequent points
 IF NOT IsFirstPoint AND (ActualLength < MaxArrayLength) THEN
 IF (MoveDir > 0 AND TriggerSource >= NextTarget) OR
 (MoveDir < 0 AND TriggerSource <= NextTarget) THEN
 dataArray[ActualLength] := SensorValue;
 ActualLength := ActualLength + 1;
 NextTarget := NextTarget + StepInterval * MoveDir;
 END_IF
 END_IF

 // Full check
 IF ActualLength >= MaxArrayLength THEN
 bRunning := FALSE; bDone := TRUE;
 bError := TRUE; ErrorCode := ERR_FULL;
 END_IF
END_IF

END_FUNCTION_BLOCK
```


### ## Template 2: Processing/Fitting Function Block



```

```iecst
FUNCTION_BLOCK FB_DataProcessor

VAR_INPUT

```


```

```

 bExecute : BOOL; // Trigger calculation
 bReset : BOOL;
 DataLength : UINT; // Valid data length
 SmoothWindow : UINT := 3; // Moving average window (odd)
 END_VAR

VAR_OUTPUT
 bDone : BOOL;
 bValid : BOOL;
 bError : BOOL;
 ErrorCode : UINT;
 Result : LREAL; // Calculated result (e.g., slope)
END_VAR

VAR
 LastExecute : BOOL;
 i : INT;
 j : INT;
 Sum : LREAL;
 Half : INT;
 StartIdx : INT;
 EndIdx : INT;
END_VAR

VAR CONSTANT
 ERR_OK : UINT := 0;
 ERR_DATA : UINT := 1; // Insufficient data
 ERR_CALC : UINT := 2; // Calculation error
END_VAR

VAR_IN_OUT
 SrcArray : ARRAY[0..9999] OF LREAL;
 OutArray : ARRAY[0..9999] OF LREAL;
END_VAR

// Reset
IF bReset THEN
 bDone := FALSE; bValid := FALSE; bError := FALSE;
 ErrorCode := ERR_OK; Result := 0;
 LastExecute := FALSE;
 RETURN;
END_IF

// Edge-triggered execution
IF bExecute AND NOT LastExecute THEN
 bDone := FALSE; bValid := FALSE; bError := FALSE;

 // Validate
 IF DataLength < 2 THEN
 bError := TRUE; ErrorCode := ERR_DATA;
 LastExecute := bExecute;
 RETURN;
 END_IF

 // --- Calculate result (e.g., slope) ---
 Result := (SrcArray[DataLength-1] - SrcArray[0]) / (DataLength - 1);

 // --- Generate output (e.g., moving average) ---
 Half := INT#(SmoothWindow - 1) / 2;
 FOR i := 0 TO INT#(DataLength - 1) DO
 StartIdx := i - Half;
 EndIdx := i + Half;
 IF StartIdx < 0 THEN StartIdx := 0; END_IF
 IF EndIdx > INT#(DataLength - 1) THEN EndIdx := INT#(DataLength - 1); END_IF
 Sum := 0;
 END FOR

```

```

 FOR j := StartIdx TO EndIdx DO
 Sum := Sum + SrcArray[j];
 END_FOR
 OutArray[i] := Sum / (EndIdx - StartIdx + 1);
 END_FOR

 bDone := TRUE; bValid := TRUE;
END_IF

LastExecute := bExecute;

END_FUNCTION_BLOCK
...

Template 3: Test Program

```iecst
PROGRAM PRG_Test

VAR
    MyFB      : FB_DataAcquisition;
    MyProc    : FB_DataProcessor;

    arrData   : ARRAY[0..9999] OF LREAL;
    arrOut    : ARRAY[0..9999] OF LREAL;

    btnStart  : BOOL;
    btnReset  : BOOL;
    btnCalc   : BOOL;

    // Status
    bRunning  : BOOL;
    bDone     : BOOL;
    nCount    : UINT;
    rResult   : LREAL;
END_VAR

// Acquisition
MyFB.bEnable := btnStart;
MyFB.bReset  := btnReset;
MyFB.SensorValue := rSensorInput;
MyFB.TriggerSource := rTriggerInput;
MyFB.MaxArrayLength := 500;
MyFB.StepInterval := 10.0;
MyFB(DataArray := arrData);

bRunning := MyFB.bRunning;
bDone := MyFB.bDone;
nCount := MyFB.ActualLength;

// Processing
MyProc.bExecute := btnCalc;
MyProc.bReset := btnReset;
MyProc.DataLength := nCount;
MyProc.SmoothWindow := 3;
MyProc(SrcArray := arrData, OutArray := arrOut);

rResult := MyProc.Result;

IF btnReset THEN
    btnCalc := FALSE;
END_IF

END_PROGRAM
...

```


A.3 references/cam-table-patterns.md

Cam Table Design Patterns and Algorithms

1. Cam Table Fundamentals

A cam table maps a master axis position to a slave axis position:

- X-axis = Master (encoder position, or winding axis angle)
- Y-axis = Slave (desired output position)

In CODESYS, cam tables are used with:

- `MC\_CamIn` – activates electronic cam coupling
- `MC\_CamTableSelect` – selects a cam table
- CAM Editor – visual curve design tool

2. Data Acquisition Patterns

Pattern A: Angle-Triggered Capture

- **Trigger source**: Winding axis angle (Y-axis feedback)
- **Captured value**: Encoder position (X-axis)
- **Interval**: Every N degrees of winding axis rotation
- **Use case**: Map winding angle → material feed position

Pattern B: Position-Triggered Capture

- **Trigger source**: Encoder position (X-axis)
- **Captured value**: Winding axis angle (Y-axis)
- **Interval**: Every N units of encoder travel
- **Use case**: Map material feed → required winding angle

Pattern C: Time-Triggered Capture

- **Trigger source**: Timer/PLC scan count
- **Captured value**: Both encoder and angle
- **Use case**: Diagnostic logging, not for cam table generation

3. Bidirectional Detection

When the trigger source can move in both directions:

```
``iecst
// Detect direction
IF TriggerSource > LastTrigger THEN
    MoveDir := 1.0; // Forward
ELSIF TriggerSource < LastTrigger THEN
    MoveDir := -1.0; // Reverse
END_IF

// Check trigger condition (both directions)
IF (MoveDir > 0 AND TriggerSource >= NextTarget) OR
   (MoveDir < 0 AND TriggerSource <= NextTarget) THEN
    // Capture data point
END_IF
``
```

4. Curve Fitting Algorithms

4.1 Average Slope (Linear Fit)

```
...
AvgSlope = (X[end] - X[0]) / (Y[end] - Y[0])
...
```

Interpretation: For every 1 degree of winding axis rotation, the encoder moves by AvgSlope units.

Linear reference curve:

```
...
```

```

FitX[i] = X[0] + AvgSlope * (Y[i] - Y[0])
FitY[i] = Y[i]
```

```

### ### 4.2 Moving Average Smoothing

Window size N (must be odd: 3, 5, 7):

```

```
Half = (N - 1) / 2
FitX[i] = average(X[i-Half .. i+Half]) // clipped at array boundaries
FitY[i] = average(Y[i-Half .. i+Half])
```

```

- N=3: light smoothing, preserves detail
- N=5: moderate smoothing, good for noisy signals
- N=7: heavy smoothing, may introduce lag

### ### 4.3 Exponential Smoothing (Alternative)

```

```
alpha = 0.3 // smoothing factor (0..1), higher = less smoothing
FitX[0] = X[0]
FitX[i] = alpha * X[i] + (1 - alpha) * FitX[i-1]
```

```

## ## 5. Array Design Considerations

- **Compile-time max**: 10000 elements (practical limit for memory)
- **Runtime effective**: controlled by `MaxArrayLength` input
- **Memory**: each LREAL = 8 bytes; 10000 \* 2 arrays \* 8 = 160KB per FB instance
- **Scan time**: processing 10000 points in one cycle may take 1-5ms depending on CPU

## ## 6. Integration with CODESYS SoftMotion

```

```iecst
// Connect to SoftMotion axis
CamGen.EncoderPosition := Axis_Encoder.Position;
CamGen.WindingAxisAngle := Axis_Winding.Position;

// After fitting, use in MC_CamIn
// Method A: Export to CSV, import to CAM Editor
// Method B: Directly use arrays with MC_CamTableSelect
```

```

## ## 7. Quality Metrics

After fitting, evaluate curve quality:

- **R<sup>2</sup>** (coefficient of determination): how well linear fit matches data
- **Max deviation**: largest difference between original and fitted
- **Standard deviation**: spread of residuals
- **Smoothness**: second derivative magnitude (lower = smoother)

These can be added as additional FB features if needed.

## A.4 references/st-code-snippets.md

```
ST Code Snippets and Utilities

1. Edge Detection (Rising/Falling Edge)

```iecst
// Rising edge
IF bSignal AND NOT bLastSignal THEN
    // Action on rising edge
END_IF
bLastSignal := bSignal;

// Falling edge
IF NOT bSignal AND bLastSignal THEN
    // Action on falling edge
END_IF
```

2. Array Initialization

```iecst
// Clear array
FOR i := 0 TO 9999 DO
    arrData[i] := 0.0;
END_FOR

// Initialize with specific value
FOR i := 0 TO 9999 DO
    arrData[i] := 1.0;
END_FOR
```

3. Moving Average (Generic)

```iecst
FUNCTION_BLOCK FB_MovingAverage
VAR_INPUT
    rInput    : LREAL;
    nWindow   : UINT := 5;
END_VAR
VAR_OUTPUT
    rOutput   : LREAL;
END_VAR
VAR
    arrBuf    : ARRAY[0..99] OF LREAL;
    nIdx      : UINT;
    nCount    : UINT;
    rSum      : LREAL;
    i         : UINT;
END_VAR

// Add new value
arrBuf[nIdx] := rInput;
nIdx := (nIdx + 1) MOD nWindow;
IF nCount < nWindow THEN
    nCount := nCount + 1;
END_IF

// Calculate average
rSum := 0;
FOR i := 0 TO nCount - 1 DO
    rSum := rSum + arrBuf[i];
END_FOR
rOutput := rSum / nCount;
```

```

```

4. Clamp / Limit

```iecst
// Clamp value between min and max
rClamped := LIMIT(rMin, rValue, rMax);

// Or manual
IF rValue < rMin THEN
    rClamped := rMin;
ELSIF rValue > rMax THEN
    rClamped := rMax;
ELSE
    rClamped := rValue;
END_IF
```

5. State Machine Pattern

```iecst
VAR
    nState : INT := 0;
END_VAR

CASE nState OF
    0: // IDLE
        IF bStart THEN
            nState := 1;
        END_IF

    1: // RUNNING
        // Do work
        IF bComplete THEN
            nState := 2;
        ELSIF bError THEN
            nState := 99;
        END_IF

    2: // DONE
        // Wait for reset
        IF bReset THEN
            nState := 0;
        END_IF

    99: // ERROR
        IF bReset THEN
            nState := 0;
        END_IF
END_CASE
```

6. Timer / Debounce

```iecst
VAR
    tonDelay : TON;
    bDelayed : BOOL;
END_VAR

// 100ms delay
tonDelay(IN := bSignal, PT := T#100MS);
bDelayed := tonDelay.Q;
```

```

## ## 7. Type Conversion

```
```iecst
// INT to LREAL
rValue := LREAL#nIntValue;

// LREAL to INT (truncates)
nValue := INT#rValue;

// Using TO_ functions
rValue := TO_LREAL(nIntValue);
nValue := TO_INT(rValue);
```
```

## ## 8. Modular Arithmetic for Circular Buffers

```
```iecst
// Circular buffer index
nNextIdx := (nCurrentIdx + 1) MOD nBufferSize;

// Wrap angle to 0..360
IF rAngle >= 360.0 THEN
    rAngle := rAngle - 360.0;
ELSIF rAngle < 0.0 THEN
    rAngle := rAngle + 360.0;
END_IF
```
```

## ## 9. Error Code Pattern

```
```iecst
VAR CONSTANT
    ERR_OK       : UINT := 0;
    ERR_PARAM    : UINT := 1;
    ERR_OVERFLOW : UINT := 2;
    ERR_TIMEOUT  : UINT := 3;
    ERR_COMM     : UINT := 4;
END_VAR

// Set error
bError := TRUE;
ErrorCode := ERR_PARAM;

// Clear error
bError := FALSE;
ErrorCode := ERR_OK;
```
```

## ## 10. VAR\_IN\_OUT Best Practices

- Use for large arrays to avoid memory copy
- Caller must declare the array and pass it
- FB reads and writes directly to the caller's array
- Array size in FB declaration must match or exceed caller's

```
```iecst
// FB declaration
VAR_IN_OUT
    DataArray : ARRAY[0..9999] OF LREAL;
END_VAR

// Caller declaration
VAR
    MyData : ARRAY[0..9999] OF LREAL;
END_VAR
```

```
// Caller invocation
MyFB(DataArray := MyData);
```
```

# 附录 B：mermaid-diagram 技能完整内容

以下为 mermaid-diagram 技能的 SKILL.md 完整内容，仅供参考。该技能可通过市场直接安装，无需手动创建。

## B.1 SKILL.md

```

name: mermaid-diagram
description: Generate Mermaid diagrams for product managers and developers. Use when: (1) creating flowcharts

Mermaid Diagram Generator

Generate valid Mermaid diagram code from text descriptions.

Diagram Type Selection

图标	名称	Mermaid 语法
[矩形] / [圆角矩形]	Flowchart	`flowchart TD`
[矩形] / API [矩形]	Sequence	`sequenceDiagram`
[矩形] / [圆角矩形]	Flowchart / C4	`flowchart LR`
[矩形] / [圆角矩形]	State	`stateDiagram-v2`
[矩形] / [圆角矩形]	Mindmap	`mindmap`
[矩形]	ER	`erDiagram`
[矩形]	Timeline	`timeline`
[矩形]	Journey	`journey`

Output Rules

1. Always wrap output in ```mermaid``` code blocks
2. Use Chinese labels when the user writes in Chinese
3. Add comments (`%%`) for complex diagrams to explain sections
4. Keep node names short (≤10 chars) to avoid layout issues
5. Validate syntax mentally before outputting – common mistakes:
 - Missing quotes around labels with spaces/Chinese
 - Wrong arrow syntax (`->` vs `-->` vs `->>`)
 - Unclosed brackets

Quick Syntax Reference

...

flowchart TD
 A[] --> B[]
 B --> C[]
 B --> D[]
 C --> D

...

sequenceDiagram
 participant A as []
 participant B as []
 participant C as []
 participant D as []
 A->>B
 B->>C
 C->>D
 D->>A

...

mindmap
 root(())
 A
 1
```

   2  
  B  
 ...

## ## Reference Files

```
- **references/diagram-patterns.md** - ████████████████████████████████
```

Read when: user asks for a specific diagram type or needs a complex multi-node diagram.



## B.2 references/diagram-patterns.md

```
Mermaid ██████
```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

— — —

```
1. ■■■■■■■■
```

```

graph TD
 subgraph UX ["UX"]
 A1[Web]
 A2[]
 A3[API]
 end
 end
 subgraph Product ["Product"]
 B1[A]
 B2[B]
 B3[C]
 end
 end
 subgraph AI ["AI/ "]
 C1[]
 C2[]
 C3[]
 end
 end
 subgraph Infra ["Infra"]
 D1[]
 D2[]
 D3[]
 end
 end
 UX --> Product
 Product --> AI
 AI --> Infra

```

— — —

## 2. ■■■■■

```

```mermaid
journey
    title 
    section 
        : 
        : 
    section 
        : 
        : 
        : 
    section 
        :  , 
        : 
    section 
        : 
        : 
```

```

— — —

### ## 3.

```
```mermaid
flowchart TD
```

```

Start([■■■■]) --&gt; A[■■■■■■]
A --&gt; B{■■■■■?}
B -- ■ --&gt; C[■■■■■]
C --&gt; D[■■■■■] --&gt; B
B -- ■ --&gt; E{■■■■■}
E -- ■■■ --&gt; F[■■403]
E -- ■■■ --&gt; G[■■■■■■■]
G --&gt; H{■■■■■?}
H -- ■ --&gt; I[■■■■■]
H -- ■ --&gt; J[■■■■■■■]
J --&gt; K[■■■■■■■]
I --&gt; End([■■])
K --&gt; End
...

```

4. ■■■■■■■■

```

```mermaid
sequenceDiagram
 participant U as ■■
 participant FE as ■■
 participant BE as ■■
 participant AI as AI■■
 participant DB as ■■■

 U->>FE: ■■■■
 FE->>BE: API■■
 BE->>DB: ■■■■■■
 DB-->>BE: ■■■■■■
 BE->>AI: ■■■■(+■■■)
 AI-->>BE: ■■■■
 BE-->>FE: ■■■■
 FE-->>U: ■■■■

 Note over BE,DB: ■■■■■■■■
...

```

---

## 5. ■■■■■■■■■■

```

```mermaid
flowchart LR
    Input[■■■■■] --&gt; Recall{■■■■}
    Recall --&gt; R1[■■■■■]
    Recall --> R2[■■■■■]
    Recall --> R3[■■■■■]
    R1 & R2 & R3 --&gt; Merge[■■■■■]
    Merge --&gt; Rank[■■\n■■■■■]
    Rank --&gt; Rerank[■■\n■■■■■]
    Rerank --&gt; Filter{■■■■■}
    Filter -- ■■/■■/■■ --&gt; Result[■■■■■]
...

```

6. ■■■■■■

```

```mermaid
stateDiagram-v2
 [*] --> ■■■
 ■■■ --> ■■■■ : ■■■■■■■■
 ■■■■ --> ■■■■ : ■■/■■

```

```

 ■■■■ --> ■■■■ : 7■■■■
 ■■■■ --> ■■■■ : ■■■■
 ■■■■ --> ■■■■ : ■■■■
 ■■■■ --> ■■■■ : 30■■■■
 ■■■■ --> ■■■■ : ■■■■
 ■■■■ --> [*] : ■■■■
 ...

```

---

## 7. ■■■■

```

```mermaid
mindmap
  root((■■■■))
    ■■■■
      ■■/■■
      ■■■■
      ■■■■
      ■■■■
      ■■■■A
        ■■■1
        ■■■2
      ■■■■B
      ■■■■
      ■■■■
      ■■■■
      ■■
      ■■■■
      ■■■■
  ...

```

8. ■■■■■■Roadmap■

```

```mermaid
timeline
 title ■■■■ 2026
 Q1 : MVP■■
 : ■■■■■■
 Q2 : ■■■■
 : AI■■■■
 Q3 : ■■■■
 : ■■■■■■
 Q4 : ■■■■
 : ■■■■
 ...

```